

EvoArena — Project Progress Report 2

Rohan Upadhyay | CSCI 411: Senior Seminar I | March 25, 2026

GitHub: <https://github.com/RyanX5/evoarena>

1. Project Overview

EvoArena is an agent-based evolutionary simulation where autonomous agents compete in a 2D arena and develop combat and survival behaviors through evolution. Each agent is controlled by a small neural network whose weights are shaped over generations using a genetic algorithm. Successful agents reproduce, passing their strategies to the next generation.

The intended use case is demonstrating how complex behaviors can emerge from simple agents under selection pressure, without any hand-coded rules. The project covers core CS concepts including neural computation, evolutionary algorithms, agent-based modeling, and system design.

Current stage: Post-midterm milestone. All core simulation systems have been operational since the midterm. This report documents four new additions: real-time fitness tracking and visualization, crossover-based reproduction, physical obstacle collision detection, and alpha agent identification.

2. Implementation Status

All modules from the midterm remain complete. The following table reflects the current state of the project including additions made since the midterm.

Module	Status	Description
<code>simulation/arena.py</code>	Complete	Arena environment, step loop, obstacle placement, agent management
<code>simulation/agent.py</code>	Complete	Neural-network-driven decisions, combat, fitness tracking, obstacle collision
<code>simulation/neural_network.py</code>	Complete	8→12→4 feedforward NN, forward pass, weight serialization
<code>visualization/visualizer.py</code>	Complete	Real-time Pygame rendering, health bars, HUD, alpha agent highlight
<code>visualization/fitness_graph.py</code>	Complete	Per-generation fitness tracking, saved to <code>fitness.png</code> via matplotlib
<code>evolution/evolve.py</code>	Complete	Elitist selection, Gaussian mutation, crossover reproduction
<code>main.py</code>	Complete	Generation loop, arena reset, visualizer integration, fitness data collection

3. Key Implementations Since the Midterm

3.1 Fitness Graph

The most significant observability gap in the simulation was the absence of any quantitative fitness tracking. Behavioral changes were visible in the arena such wall hugging, center clustering, strategy divergence, but there was no way to measure whether the population was actually improving across generations.

A new module, `visualization/fitness_graph.py`, was added to address this. At the end of each generation, best and average fitness are computed from the surviving agent pool and appended to two running lists in `main.py`:

```
best_fitness = max(a.fitness() for a in arena.agents)
avg_fitness = sum(a.fitness() for a in arena.agents) / len(arena.agents)

best_fitness_history.append(best_fitness)
avg_fitness_history.append(avg_fitness)
graph.update(best_fitness_history, avg_fitness_history)
```

The `FitnessGraph` class uses matplotlib with the `Agg` backend (non-interactive, no GUI window) to avoid threading conflicts with Pygame. It renders the graph to `fitness.png` each generation, which can be viewed alongside the arena in any auto-refreshing image viewer.

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

```

class FitnessGraph:

    def __init__(self, path: str = "fitness.png"):
        self.path = path

    def update(self, best_history: list, avg_history: list):
        fig, ax = plt.subplots(figsize=(7, 4))
        generations = list(range(1, len(best_history) + 1))
        ax.plot(generations, best_history, color="#ffdc32", linewidth=2, label="Best")
        ax.plot(generations, avg_history, color="#50c8c8", linewidth=2, label="Avg")
        fig.savefig(self.path, facecolor="#0f0f19", dpi=120)
        plt.close(fig)

```

The graph plots best fitness in yellow and average fitness in cyan across all generations run so far. The Agg backend was chosen specifically because TkAgg and other interactive backends conflict with Pygame's ownership of the main thread, causing a fatal GIL error at runtime.

3.2 Crossover Reproduction

Prior to this milestone, the evolution engine used single-parent reproduction exclusively which meant that one elite was selected, its weights were mutated with Gaussian noise, and the result became a child agent. While this worked, it limited the evolution engine to local exploration of the weight space: each child could only drift from one parent.

Crossover was added to `evolution/evolve.py` to enable two-parent reproduction. When at least two elites are available, two parents are selected and their weight vectors are mixed using uniform crossover i.e. each weight in the child is independently drawn from either parent A or parent B with equal probability:

$$\text{child}_i = \begin{cases} w_i^A & \text{if } m_i > 0.5 \\ w_i^B & \text{if } m_i \leq 0.5 \end{cases}, \quad m_i \sim \mathcal{U}(0, 1)$$

Gaussian mutation is applied on top of the crossover result, identical to the pure mutation path:

$$\mathbf{w}' = \mathbf{w}_{\text{child}} + \mathcal{N}(0, \sigma^2 \mathbf{I}), \quad \sigma = 0.1$$

```
while len(mutated_agents) < config.POPULATION_SIZE:

    if len(elites) >= 2:
        parent_a = random.choice(elites)
        parent_b = random.choice([e for e in elites if e is not parent_a])
        weight_a = parent_a.brain.get_weights()
        weight_b = parent_b.brain.get_weights()

        mask = np.random.rand(len(weight_a)) > 0.5
        child_weight = np.where(mask, weight_a, weight_b)
    else:
        elite = random.choice(elites)
        child_weight = elite.brain.get_weights()

    mutated = child_weight + np.random.normal(0, config.MUTATION_RATE, size=c
```

```
new_brain = NeuralNetwork(weights=mutated)
mutated_agents.append(Agent(x=0, y=0, brain=new_brain))
```

The fallback to single parent mutation when fewer than two elites exist prevents a crash in edge cases where the population collapses.

3.3 Obstacle Collision Detection

Since Report 1, the five static obstacles in the arena have been purely visual. Agents passed through them without any physical interaction. This was an intentional deferral to keep the foundation milestone simple, but it left a gap between the rendered arena and the actual simulation environment.

Collision detection was added via a new `handle_obstacles()` method in `agent.py`. The method uses AABB (axis-aligned bounding box) vs. circle collision detection. For each obstacle, it finds the closest point on the rectangle to the agent center using clamping, checks whether the distance is less than `AGENT_RADIUS`, and if so pushes the agent out along the collision normal and reflects its velocity:

```
def handle_obstacles(self, obstacles: list):
    for obs in obstacles:
        closest_x = max(obs.x, min(self.x, obs.x + obs.width))
        closest_y = max(obs.y, min(self.y, obs.y + obs.height))

        dx = self.x - closest_x
        dy = self.y - closest_y
        distance = np.sqrt(dx**2 + dy**2)
```

```

if distance < AGENT_RADIUS:
    if distance == 0:
        self.y -= AGENT_RADIUS
        self.vy *= -1
        continue

    nx = dx / distance
    ny = dy / distance

    self.x = closest_x + nx * AGENT_RADIUS
    self.y = closest_y + ny * AGENT_RADIUS

    dot = self.vx * nx + self.vy * ny
    self.vx -= 2 * dot * nx
    self.vy -= 2 * dot * ny

```

The velocity reflection formula $\mathbf{v}' = \mathbf{v} - 2(\mathbf{v} \cdot \hat{n})\hat{n}$ mirrors the agent's velocity across the surface normal, producing a physically correct bounce. The edge case where `distance == 0` (agent center is fully inside an obstacle) is handled by a simple upward push to prevent a division-by-zero on the normal calculation.

`handle_obstacles()` is called in `arena.step()` immediately after `agent.move()` :

```

agent.move(self.width, self.height)
agent.handle_obstacles(self.obstacles)

```

3.4 Alpha Agent Identification

A small but useful visual addition: the highest-fitness agent alive at each step is identified as the "alpha" and rendered with a gold outline ring in the visualizer. This makes it immediately obvious which agent the current elite lineage descends from and provides a reference point for observing behavioral strategies in real time.



Fig: Alpha agent is marked with a golden color

4. Fitness Analysis

Running the simulation for 115 generations produced the fitness curves below, first under pure mutation and then with crossover added.

4.1 Pure Mutation Baseline (115 Generations)

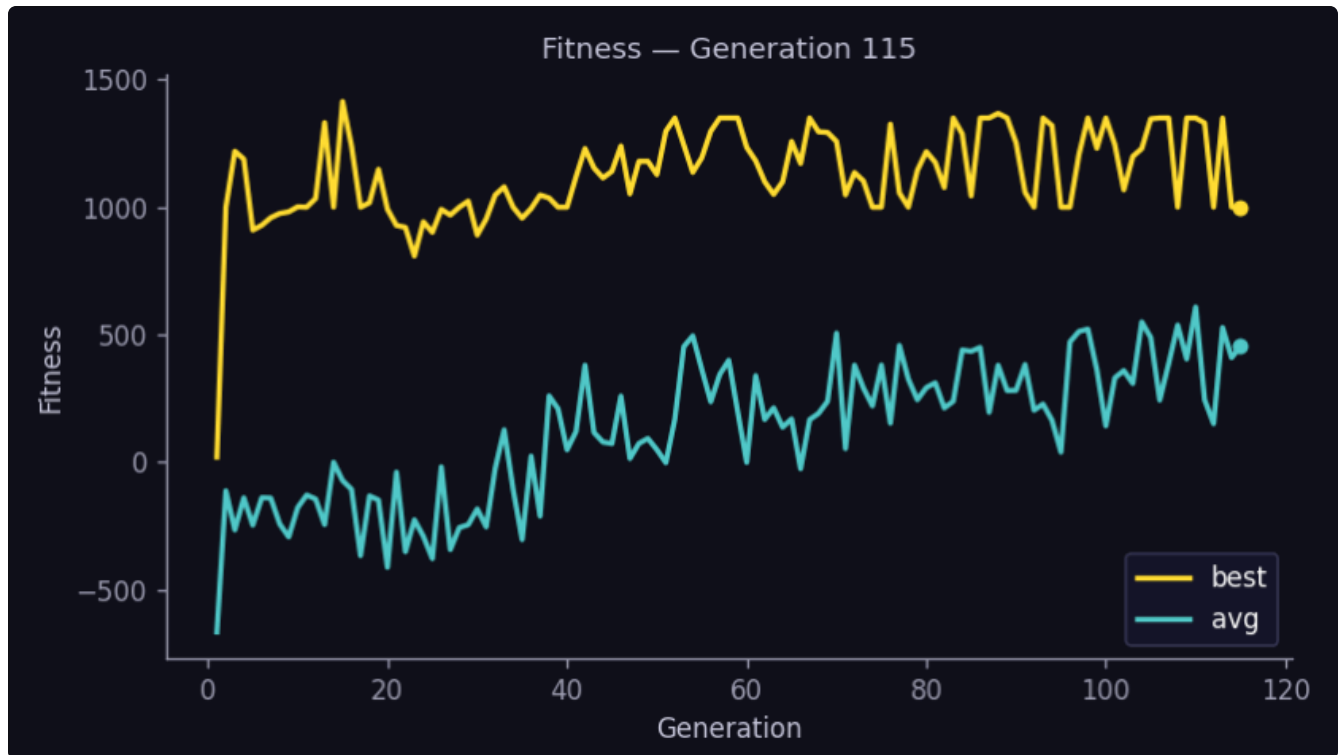


Fig 1: Fitness across 115 generations under pure mutation. Best fitness plateaus around 1000–1400 from generation 2 onward. Average fitness rises from approximately -700 to +400, reflecting population-wide elimination of wall-hugging strategies.

Several phases are visible in the curve:

Generations 1–2: Best fitness jumps from near zero to ~1200. One agent with favorable initial weights immediately finds combat, accumulates kills at 150 points each, and establishes the elite lineage.

Generations 2–35, average negative: The majority of the population remains dominated by the wall penalty. Agents accumulating large `steps_near_wall` values see the penalty term $-2.0 \cdot W$ wipe out their survival and damage contributions, producing negative total fitness. Only the elite lineage consistently scores positive.

Generations 35–115, average crossing zero: The wall-penalty-dominated strategy is gradually eliminated from the population as elite weights propagate. By generation 35, enough agents have inherited wall-avoidance behavior that the population average turns positive. This correlates directly with the center-clustering behavior documented in the midterm at generation 39.

Plateau: Best fitness oscillates between 1000–1400 for the full 115 generations without breaking through. This is the natural ceiling of single-parent mutation — the evolution engine is performing local search and has reached a local optimum it cannot escape through random perturbation alone.

4.2 Crossover Results (115 Generations)



Fig 2: Fitness across 115 generations with crossover-based reproduction. Average fitness crosses zero significantly earlier (~generation 10–15 vs. ~generation 35). Best fitness briefly exceeds 1500, a new high not reached under pure mutation.

The crossover run shows two measurable improvements over the pure mutation baseline:

Faster population improvement: Average fitness crosses zero around generation 10–15 under crossover, compared to generation 35 under pure mutation. This is the most significant difference — crossover propagates good traits from two elites simultaneously, lifting the population quality faster than single-parent drift.

Higher best fitness ceiling: Best fitness briefly exceeds 1500 during the crossover run, a value not reached in 115 generations of pure mutation. This

demonstrates crossover's ability to combine distinct adaptations from two parents into a single child.

Diminishing returns at convergence: By generation 115, the two runs converge to a similar steady state: best around 1000–1400, average around 250–400. This is expected: as the population converges and elites become increasingly similar to one another, crossover between two near-identical parents produces children barely different from either, reducing its advantage over plain mutation.

This finding is itself informative. Crossover's benefit is largest when population diversity is high, early in the run, or with a larger or more varied elite pool. Once the population has converged, the diversity required for crossover to be effective is no longer present.

5. Challenges and Solutions

5.1 Matplotlib / Pygame Threading Conflict

The initial implementation of `FitnessGraph` used `pygame.Window` for a separate live window, which requires Pygame 2.x and was unavailable on the development machine. A second attempt used matplotlib's TkAgg backend with `plt.ion()` for a non-blocking live window, but this produced a fatal GIL error at runtime: TkAgg and Pygame both attempt to own the main thread's event loop.

The solution was to switch to matplotlib's `Agg` backend, which renders entirely to memory with no GUI window, and save the output to `fitness.png` after each generation. Viewing the file in Windows Photos provides a live-ish update since

the application auto-refreshes when the file on disk changes. This approach has zero conflict with Pygame and no threading complexity.

5.2 Obstacle Collision Edge Case

The AABB vs. circle collision resolution requires computing a unit normal from the vector between the closest rectangle point and the agent center. If the agent center is exactly on the closest point (distance = 0), the normal is undefined and the division produces NaN. This can occur when an agent spawns inside or is pushed fully into an obstacle.

The edge case is handled explicitly before the normal calculation. if `distance == 0`, the agent is pushed upward by `AGENT_RADIUS` and its vertical velocity is reversed, a safe fallback that moves the agent out of the obstacle without attempting to compute a degenerate normal.

6. Timeline Comparison

The original proposal outlined the following schedule for this period:

- **Week 11–12:** Visualization improvements and comparative experiments
- **Week 13:** Testing, parameter analysis, and documentation
- **Week 14:** Final report submission, presentation, and live demonstration

Progress against plan:

All Week 11–12 targets have been addressed. The fitness graph provides the visualization and comparative data (pure mutation vs. crossover) called for in this

phase. Obstacle collision detection and crossover complete the remaining core simulation features. The alpha agent visual was an additional improvement not in the original plan.

Week 13 work is parameter sensitivity analysis and extended documentation.

7. Next Steps

Week 13:

- Parameter sensitivity analysis: systematically vary mutation rate, population size, and fitness weights and document effects on emergent behavior
- Run extended simulations (100+ generations) under varied parameters and document behavioral trends
- Save and replay champion agents from different generations for comparative analysis

Week 14:

- Final written report covering system design, algorithms, experiments, and results
 - Polish GitHub repository: clean README, inline documentation, reproducibility instructions
 - Final video presentation demonstrating multi-generation behavioral evolution
-

8. Tools Usage Acknowledgment

AI tools were used to assist with code suggestions, debugging, and documentation drafting throughout this project. All AI-assisted outputs were reviewed, modified, and integrated with full understanding of the implemented algorithms. This is documented in the GitHub repository README as required.

The flowchart figure in Appendix is made in mermaid. This report is written in Markdown and LaTeX in an online editor I made called blankt.

9. Appendix

Fig A1: EvoArena simulation flowchart (unchanged from Report 1)

